

UFUG 2602 数据结构与算法设计：课程 信息

曹北彦、常腾飞

物联网学域与智能工程学院
香港科技大学（广州）

UFUG2602：课程信息（1）

- **授课教师：常腾飞** (tengfeichang@hkust-gz.edu.cn)
- **助教：**
 - 蒋宁聪 (njiang300@connect.hkust-gz.edu.cn)
 - 曹满江 (mcao999@connect.hkust-gz.edu.cn)
- **上课时间：**
 - 讲课：周二、周四 9:00–10:20, E1-147
 - 实验：周五 17:00–17:50, E1-227 | 冯·诺依曼实验室 (W2-224D)
 - **答疑时间：周二 11:00–12:00, W2-606**

UFUG2602：课程信息（2）

- 教材：

数据结构与算法：Python 语言描述 (Data Structures & Algorithms Using Python)

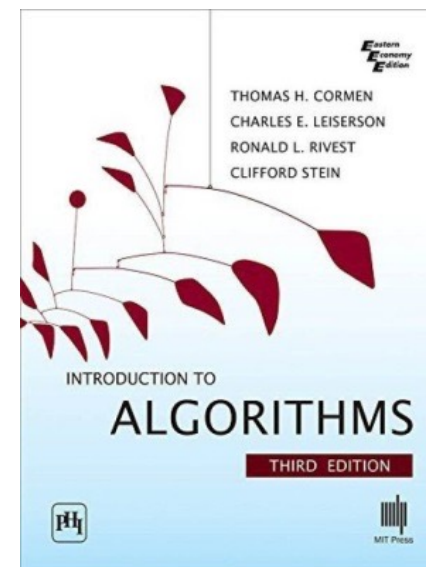
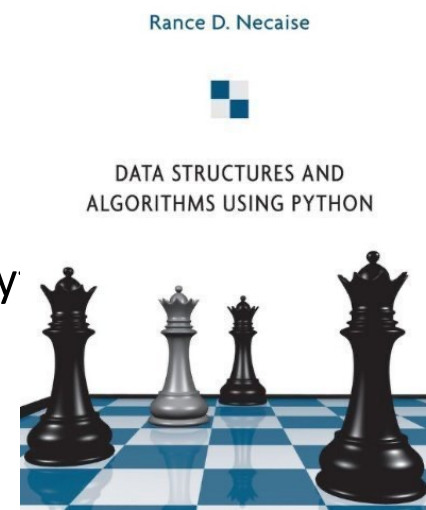
作者：Rance D. Necaise

电子书已放在 Canvas

- 更多参考书：

《算法导论》 (Introduction to Algorithms)

作者：Thomas H. Cormen、Charles E. Leiserson、
Ronald L. Rivest、Clifford Stein



课程概览

日期	课程内容
第 1 周	课程导论、抽象数据类型与数组
第 2 周	渐近复杂度与查找
第 3 周	基本排序
第 4 周	链表、栈与队列
第 5 周	递归
第 6 周	高级排序
第 7 周	散列（哈希）
第 8 周	树的基础与期中考试
第 9 周	树的遍历
第 10 周	二叉堆
第 11 周	动态二叉搜索树
第 12 周	哈夫曼编码
第 13 周	字符串（KMP 算法）
待定	期末考试

考核方式

	成绩占比	截止日期
随堂测验（实验课）	10%	随堂测验 (待定)
期中考试	35%	待定
书面作业（编程）	20%	待定
期末考试	35%	待定

UFUG2602：预期学习成果（ILOs）

1. 用 Python 实现基本的数据结构和算法
2. 使用链表、矩阵、哈希表、树等数据结构解决问题。
3. 运用贪心、分治、回溯等方法设计算法。
4. 针对给定问题选择并组合合适的数据结构与算法。

课前调查

- 以前学过数据结构或算法吗？
- 入读港科大（广州）之前学过编程吗？
- 编程学习、经历或项目中有什么困扰你的地方？
- 你希望在这门课里学到什么？

推荐的学习方式

- 目标：通过提问、讨论和讲解，促进更深入、更多样的理解
 - 课堂参与
 - 练习（实验练习与纸笔练习）
 - 及时复习（例如使用费曼学习法）
 - 多提问
- 小贴士：
 - 在做中学
 - 良好的时间管理
 - 多问自己/他人「为什么」，不要满足于单一答案（即便来自权威）

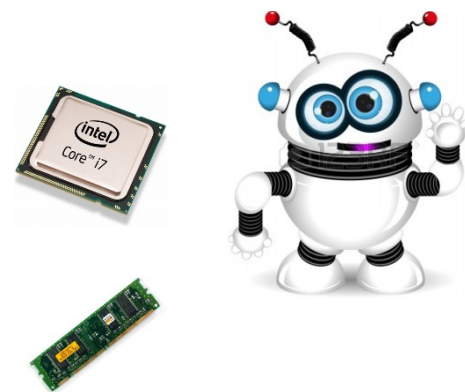
UFUG 2602 数据结构与算法设计

提纲

- 引言
 - RAM 模型
- 数据类型
 - 用户自定义数据类型
 - 抽象数据类型
- 基于数组的序列
 - Python 序列类型
 - 底层数组
 - 动态数组
 - 多维数组

什么是编程语言？

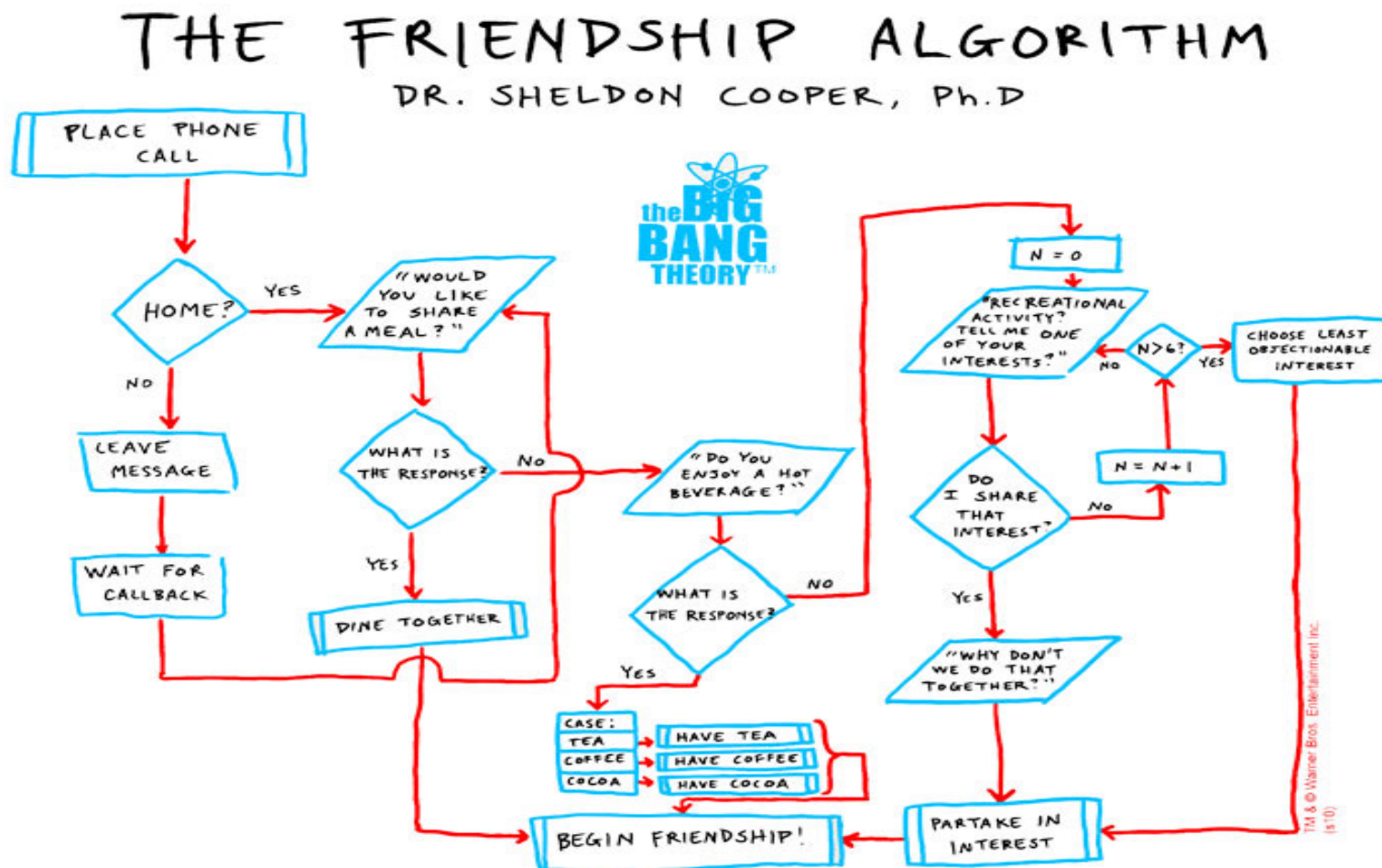
- 自然语言
 - 中文、英语、日语.....
- 编程语言 (PL)
 - 高级编程语言
 - Pascal、C、Java.....
 - 低级编程语言
 - 汇编语言
 - 由 CPU 执行的机器语言
- 编程：一步步告诉 CPU 要做什么
 - $2 + 3 = ?$
 - 输入/输出,
 - 得到答案 (计算)



数据结构与算法

- 算法：计算过程的纲要与本质，即逐步指令
- 程序：用某种编程语言对算法的实现
- 数据结构：为解决问题而对数据所作的组织

什么是算法?



算法像菜谱吗？

- 食材（输入数据）
- 厨具（计算资源）
- 步骤清单（指令序列）



算法

- 算法是一组有限的指令；按这些指令执行，就能完成特定任务、解决问题。
- 所有算法都满足以下准则：
 - **输入：提供 0 个或多个量。**
 - **输出：至少产生一个量。**
 - **确定性：每条指令都清晰、无歧义。**
 - **有穷性：在所有情况下，算法都在有限步后终止。**
 - **可行性：每条指令都必须足够基本（可执行）。**

什么是基本指令？

- 与人不同，一条指令通常只能做一件非常基本的事，
 - 读/写一个数据值
 - 比较两个数据值
 - 算术运算（+、-、 $\times$ 、 $\div$ ）等
 - 取决于计算机、编程语言……
- 以整理扑克牌为例。
 - 人可以很快把牌排好，因为能同时看见所有牌。
 - 算法做不到这一点。



计算模型

- **真实计算机很复杂**

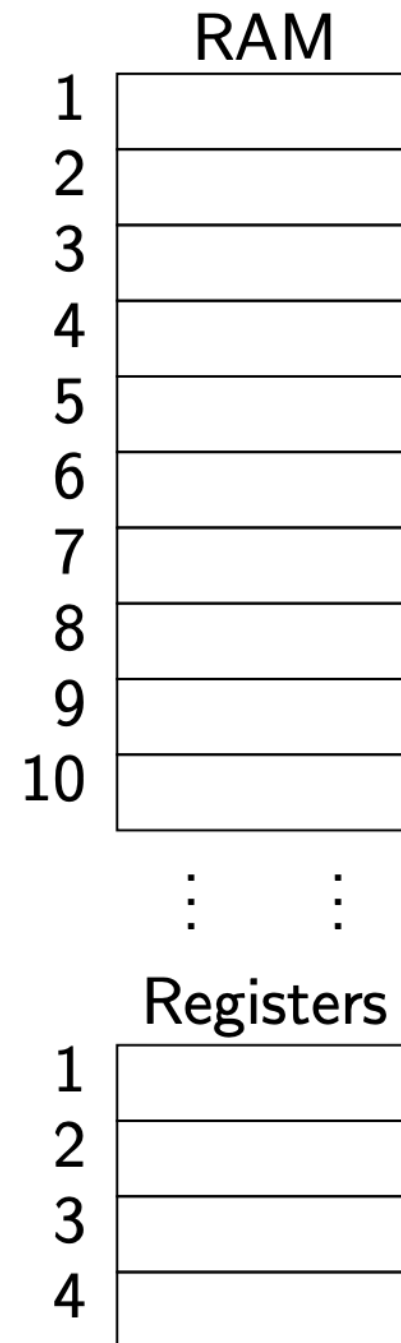
- 内存层次、浮点运算、垃圾回收器、编译器优化、不同编程语言,

- **计算模型:**

- 计算机的简单抽象
 - 定义“游戏规则”:
 - 算法允许执行哪些操作?
 - 每个操作的代价是什么?
 - 算法的代价 = Σ 其所有操作的代价

RAM 模型 (1)

- **RAM 模型：随机访问机器模型 (Random Access Machine)**
 - 无限的随机访问存储器（可看成数组），每个单元有唯一地址
 - 每个单元存放一个字（word），例如整数、字符、地址等。
 - **输入：存放在 RAM 中**
 - **输出：写回到 RAM 中**
 - 有限（常数）个寄存器（例如 4 个）
- **在一步之内可以：**
 - 把一个字从内存装入寄存器
 - 在寄存器上做运算（+、-、*、/）、位运算、比较等
 - 把一个字从寄存器写回内存



RAM 模型 (2)

- **RAM 模型中的算法**

基本操作的序列 (类似于汇编代码)

- **例子：计算两个整数之和**

假设整数存放在 $M[1]$ 和 $M[2]$

把结果写到 $M[3]$

- **算法的代价：**

运行时间：基本操作的总次数

空间：使用的内存单元总数

本节要点

- 这不是一门编程课；但你会通过编程练习来解决问题。
- 编程是软件开发的最后一步，只有在算法清楚之后才进行。
- 计算机科学是数学的一个分支，其艺术体现在算法之美。
 - 学习算法并不必须以会编程为前提。

提纲

- 引言
 - RAM 模型
- **数据类型**
 - 用户自定义数据类型
 - 抽象数据类型
- 基于数组的序列
 - Python 序列类型
 - 底层数组
 - 动态数组
 - 多维数组

数据类型

- 算法是用来操作数据的。
- 一个数据项表示现实世界中的某事物，例如学号（整数）、贷款余额（实数）等。
- 数据类型是编程语言中的概念：一组对象（数据值），以及作用在这些对象上的一组运算。
- 例如，整数类型
 - 取值：-100、0、200.....
 - 运算：+、-、/（除）、*（乘）等
- 编程语言提供整数、字符、浮点数、双精度等基本数据类型。

数据结构

- 数据结构是数据的逻辑组织方式。
 - 对学生记录而言，组成元素是各个字段，它们描述学生的不同属性。
 - 在 Python 中，常用字典和类把相关数据组织在一起。

```
student = {  
    "name": "John Doe",  
    "student_id": 1234567  
}  
  
another_student = {  
    "name": "Jane Smith",  
    "student_id": 7654321  
}
```

```
class Student:  
    def __init__(self, name, student_id):  
        self.name = name  
        self.student_id = student_id  
  
# 创建学生对象  
student_1 = Student("John Doe", 1234567)  
student_2 = Student("Jane Smith", 7654321)  
  
# 访问属性  
print(student_1.name, student_1.student_id)  
print(student_2.name, student_2.student_id)
```

用户自定义数据类型 (1)

数据结构不是数据类型!

- 问题：
 - 基本类型 (如 char、int) 不够用时, 我们能否自己定义想用的数据类型?
- 回答：
 - 除基本类型外, 我们可以把任意用户自定义数据结构 (如上页的学生例子) 视为用户自定义数据类型: 关键是为它规定一组相关操作。

用户自定义数据类型 (2)

- 问答：
 - 基本类型和用户自定义类型是否应一视同仁？
 - 原则上应该。但实际取决于编程语言。
- 对基本类型（例如整数），我们必须使用语言提供的运算。
 - 我们不必知道整数在内存中如何表示。
 - 我们也不必知道整数运算是如何实现的。
- 对用户自定义类型，我们却往往知道得太清楚！
 - 我们知道它在内存中如何表示。
 - 我们也知道操作是如何实现的。

用户自定义数据类型 (3)

- 把数据结构了解得太清楚，会有什么问题？！
- 回答：
 - 我们可能写出严重依赖具体表示与具体操作方式的程序。
 - 需要时很难改用户自定义类型的表示或实现，因为很难弄清别人如何使用这个类型。
 - 例子：
 - `student_1 = Student("John Doe", 1234567)`
 - `student_1.student_id = 7654321`
- 做法：
 - 必须隐藏细节！让使用者不可能知道过多内部信息。（非常重要）

用户自定义数据类型 (4)

- 如何隐藏细节？！
- 回答：
 - 用类来定义用户自定义数据类型。
 - 用方法操作数据，并隐藏实现细节。
 - 把属性标为私有（用 `_` 或 `__`），避免直接访问。

```
class Student:
    def __init__(self, name, student_id):
        self.__name = name          # Private attribute
        self.__student_id = student_id # Private attribute

    # Public method to access private data
    def get_student_id(self):
        return self.__student_id

    # Public method to update private data
    def set_student_id(self, new_id):
        if isinstance(new_id, int) and new_id > 0:
            self.__student_id = new_id
        else:
            raise ValueError("Invalid student ID!")
```

抽象数据类型 (ADT)

- ADT 规定可以执行哪些操作，但隐藏它们如何实现。
- 它把接口（ADT 做什么）与实现（怎么做）分离开。

ADT 示例：集合 (Set)

- 数据取值：{1, 3, 5, 8}、{5, 8, 12}.....
- 操作：
 - `search(integer, set)`：判断某整数是否在集合中
 - `intersection(set, set)`：返回两集合的交集
 - `union(set, set)`：返回两集合的并集
- 实现细节
 - 表示：数组、链表.....
 - 操作的实现：可以有很多多种方式，取决于数据表示和所用编程语言。

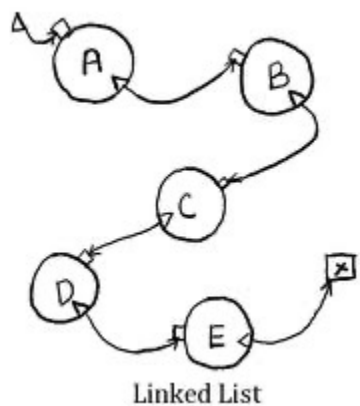
常见的 ADT 有哪些?

- 表、栈、队列、树、图等。
 - 把同类对象组织在一起
- ADT 的效率
 - 数据表示: 数据结构
 - 操作的实现: 算法

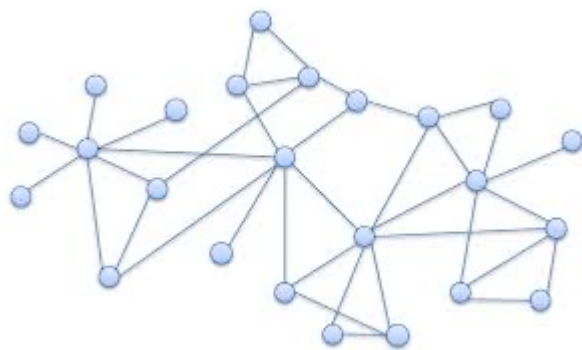
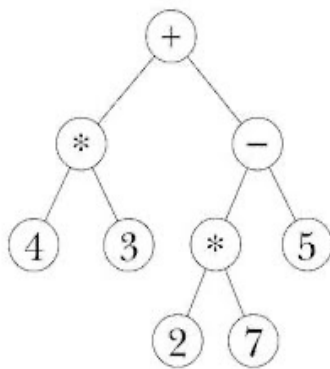
Array:

23	4	6	15	5	7
0	1	2	3	4	5

↑
Array index



Tree



数据类型小结

- 数据结构（表示/组织）+ 操作 = 数据类型
- 数据类型
 - 基本数据类型：由编程语言提供
 - 用户自定义数据类型：程序员按需定义
- 用户自定义数据类型
 - 可以建立在其他类型之上（基本类型或用户自定义类型）。
- 抽象数据类型
 - 两处分离：
 - 数据值的定义与其表示相分离。
 - 操作的定义与其实现相分离。
 - 封装：隐藏实现的内部细节。

编程语言对 ADT 的支持

- 面向对象编程语言：
 - 例如 Python、C++、Java（通过类）
- C 语言：
 - C 没有实现 ADT 的显式机制。
 - 但在 C 中仍然可以、也值得去设计 ADT

基于数组的序列

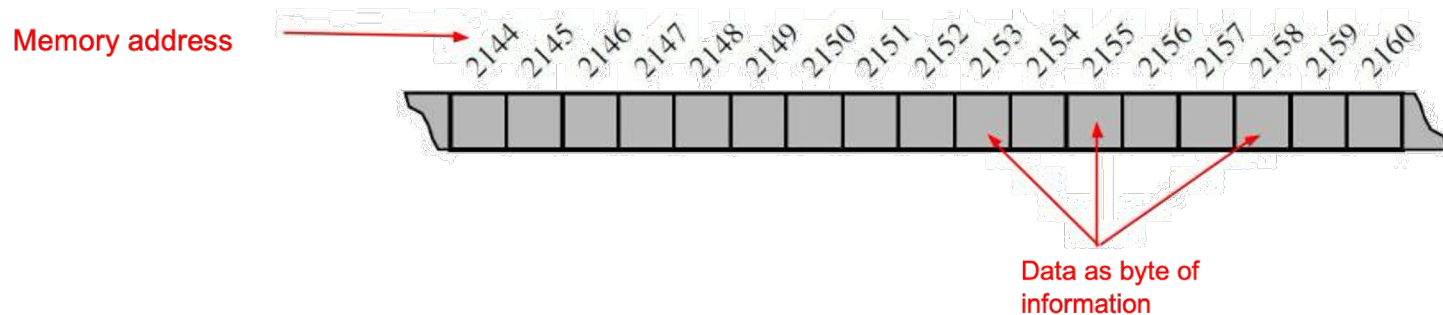
提纲

- Python 序列类型
- 底层数组
- 动态数组
- 多维数组

Python 的序列类型

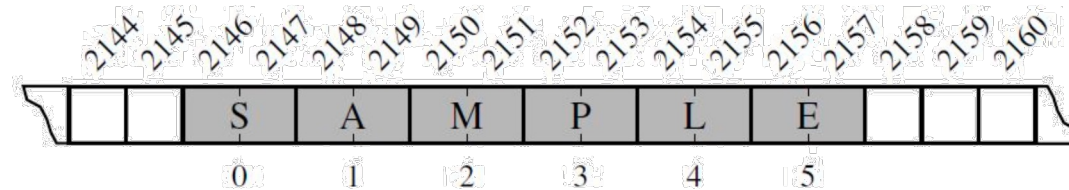
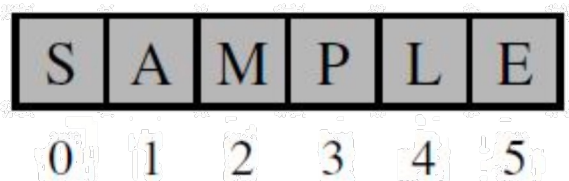
- 序列类：list、tuple 和 str
- 共同行为：
 - 都支持用下标访问单个元素，例如 `seq[k]`。
 - 底层都用称为数组的概念来表示序列。
- 区别在于：
 - 它们所表示的抽象，以及这些复杂结构如何实现。
- 深入理解这些结构，对成为优秀程序员很重要。

底层数组



- 计算机的主存由比特（bit）组成。
- 比特通常按体系结构分组为更大的单位。典型单位是字节（byte），等于 8 比特。
- 内存中每个字节都有一个唯一编号，即内存地址。
- **随机访问存储器（RAM）**
 - 尽管内存单元顺序排列，硬件仍能按地址高效访问任意位置。取第 8675309 号字节与取第 309 号字节同样容易。
- 任意单个字节都可以在 $O(1)$ 时间内存取。

- 数组是一组相关变量，在内存中连续存放。
- 例子：字符串存成单个字符的有序序列。
- 数组每个单元占用相同字节数，从而可在常数时间内访问任意单元。



- 每个 Unicode 字符用 16 位即 2 字节表示。
- 六个字符的串如 'SAMPLE' 会占用连续 12 个字节。
- 这是长度为 6 的字符数组，共需 12 字节。
- 数组中的每个位置称为单元（cell），用整数下标描述其位置。
- 例如，下标 4 的单元存放 'L'，对应内存地址 2154 和 2155。
- 下标为 k 的单元地址 = $\text{start} + \text{cellsize} * k$ 。

动态数组

- 创建底层数组时，必须在实例化时明确指定精确大小。
- Python 的 tuple 和 str 同样如此：它们不可变，一经创建，大小和内容都不能改。
- Python 的 list 类表示动态数组，可以向其中增删元素。
- 动态数组的语义
 - 它维护一块底层数组，容量大于当前列表长度。
 - 预留容量用尽时，系统会分配更大的新数组，旧数组随后被回收。

- 空列表也占空间（本机上约 64 字节），用于保存以下信息：
 - 当前实际存放的元素个数。
 - 当前最多还能存放多少个元素（容量）。
 - 指向当前已分配数组的引用。
- 在 64 位 CPU 上，每个内存地址是 64 位（8 字节）的数。
- 预留容量会随不断追加对象而增长：例如从 32 字节（4 个单元）到 64 字节（8 个单元），以此类推。

```

1 # analyzing a dynamic array
2 import sys
3 data = []
4 for k in range(27):
5     a = len(data)
6     b = sys.getsizeof(data)
7     print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a,b))
8     data.append(None)

```

Length: 0; Size in bytes: 64
Length: 1; Size in bytes: 96
Length: 2; Size in bytes: 96
Length: 3; Size in bytes: 96
Length: 4; Size in bytes: 96
Length: 5; Size in bytes: 128
Length: 6; Size in bytes: 128
Length: 7; Size in bytes: 128
Length: 8; Size in bytes: 128
Length: 9; Size in bytes: 192
Length: 10; Size in bytes: 192
Length: 11; Size in bytes: 192
Length: 12; Size in bytes: 192
Length: 13; Size in bytes: 192
Length: 14; Size in bytes: 192
Length: 15; Size in bytes: 192
Length: 16; Size in bytes: 192
Length: 17; Size in bytes: 264
Length: 18; Size in bytes: 264
Length: 19; Size in bytes: 264
Length: 20; Size in bytes: 264
Length: 21; Size in bytes: 264
Length: 22; Size in bytes: 264
Length: 23; Size in bytes: 264
Length: 24; Size in bytes: 264
Length: 25; Size in bytes: 264
Length: 26; Size in bytes: 344

32 bytes are reserved for storing 4 object references.

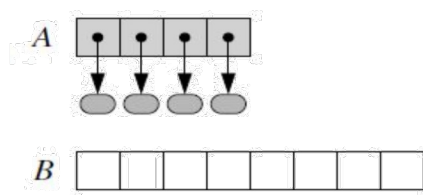
Space for 4 object references

Space for 8 object references stored.

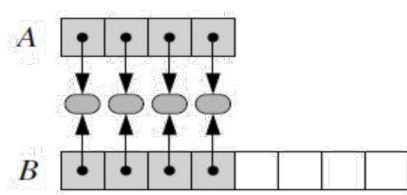
Space for 9 object references stored.

实现动态数组

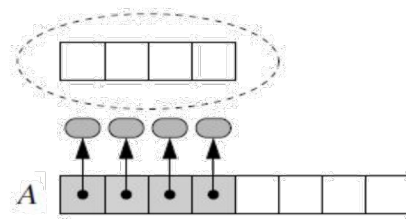
- 关键是要能扩大存放列表元素的数组 A。
- 底层数组已满时再追加元素，执行以下步骤：
 - 分配容量更大的新数组 B。
 - 令 $B[i] = A[i]$, $i = 0, \dots, n-1$, 其中 n 为当前元素个数。
 - 令 $A = B$, 即改用 B 作为列表的底层数组。
 - 把新元素插入新数组。



(a)

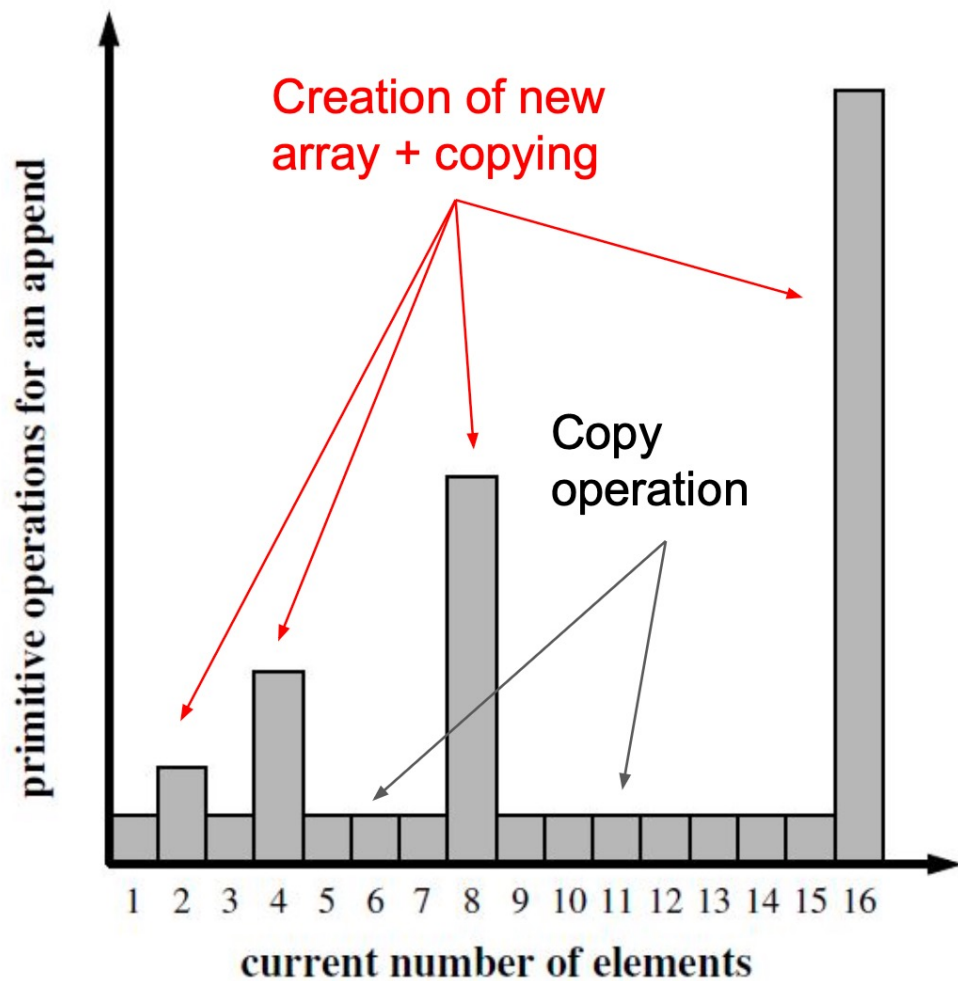


(b)



(c)

- 新数组应该开多大？
 - 常见规则：新数组容量取已满旧数组的两倍。



Run-time of a series of append operations on a dynamic array

```

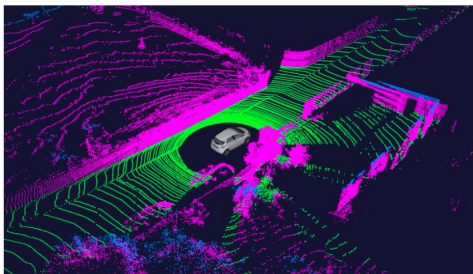
1 # Implementing a dynamic array
2 import ctypes
3 class DynamicArray:
4     ''' A dynamic array class akin to a simplified Python list '''
5     def __init__(self):
6         ''' create an empty array '''
7         self._n = 0                # count actual elements
8         self._capacity = 1         # default array capacity
9         self._A = self._make_array(self._capacity) # low-level array
10
11     def __len__(self):
12         ''' return number of elements stored in the array'''
13         return self._n
14
15     def __getitem__(self, k):
16         ''' return element at index k '''
17         if not 0 <= k < self._n:
18             raise IndexError('invalid index')
19         return self._A[k]
20
21     def append(self, obj):
22         ''' Add object to the end of the array'''
23         if self._n == self._capacity: # no enough room
24             self._resize(2*self._capacity) # so double the capacity
25         self._A[self._n] = obj
26         self._n += 1
27
28     def _resize(self, c):
29         ''' Resize the internal array to capacity c'''
30         B = self._make_array(c) # new (bigger) array
31         for k in range(self._n):
32             B[k] = self._A[k] # copy data from old array into new array
33         self._A = B # use the bigger array
34         self._capacity = c
35
36     def _make_array(self, c): # nonpublic utility
37         ''' Return new array with capacity c'''
38         return (c * ctypes.py_object)()
39
40 A = DynamicArray()
41 A.append(1)
42 A.append(2)
43 A.append(7)
44 print(A[0], A[1], A[2])

```

1 2 7

多维数据集

- 具有多于一个维度。二维列表可称为矩阵。
- 例子：
 - 地理信息可以自然地用二维表示。
 - 激光雷达得到的距离数据是三维数据。
 - 灰度图像是二维图像。



数组

	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

Two-dimensional integer dataset (Number of stores in various regions in district)

```
22  18  709  5  33
45  32  830 120 750
4   880  45  66  61
```

```
Data = [ [22,18, 709, 5, 33], [45, 32, 830, 120, 750],
          [4, 880, 45, 66, 61] ]
```

Each element can be accessed by using a syntax:
`data[1][3]`

构造多维列表

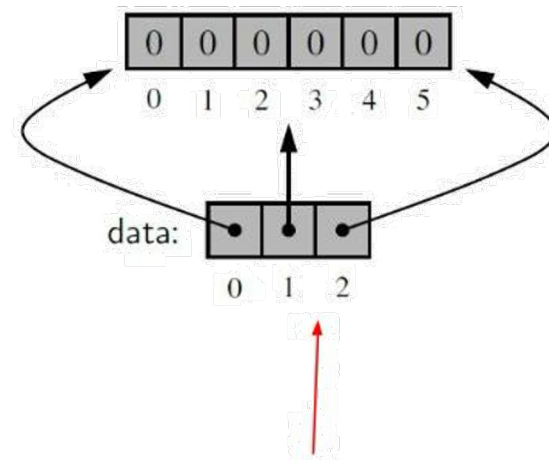
- 快速初始化一维列表:

`Data = [0] * n`

- 初始化多维列表:

`Data = ( [0] * c ) * r`

`Data = [ [0] * c ] * r`



Creates 1-D list

Warning: this is a mistake

Warning: still a mistake

The problem is that all `r` entries of the list known as `data` are references to the same instance of a list of `c` zeros

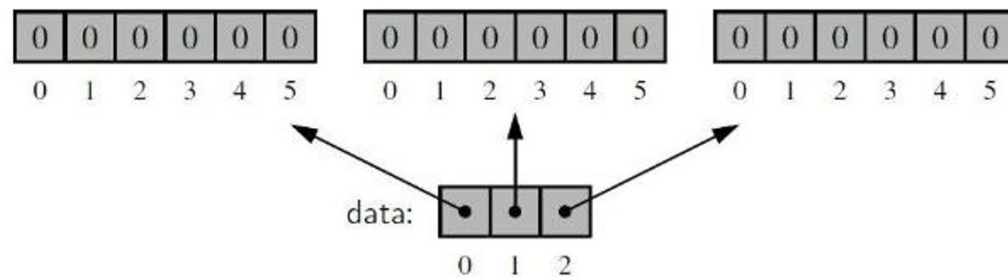
```
1 data = [[0]*3]*3
2 print(data)
3 data[2][0] = 5
4 print(data)
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
[[5, 0, 0], [5, 0, 0], [5, 0, 0]]
```

初始化二维列表的正确写法

`data = [[0] * c for j in range(r)]`

- 这样才能保证主列表的每个单元都指向独立的子列表实例。



```
1 # data = [[0]*3]*3
2 # print(data)
3 # data[2][0] = 5
4 # print(data)
5
6 data2 = [[0]*4 for j in range(3)]
7 print(data2)
8
9 data2[2][0] = 5
10 data2[0][1] = 2
11 print(data2)
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
[[0, 2, 0, 0], [0, 0, 0, 0], [5, 0, 0, 0]]
```

```

1 # Tic-Tac-Toe Game
2 class TicTacToe:
3     '''
4     Management of a Tic-Tac-Toe Game
5     No strategy
6     '''
7     def __init__(self):
8         ''' start a new game'''
9         self._board = [ [' ' ] * 3 for j in range(3) ]
10        self._player = 'X'
11
12    def mark(self, i, j):
13        ''' Put X or O mark at position (i,j) for next player's turn '''
14
15        if not(0 <= i <= 2 and 0 <= j <= 2):
16            raise ValueError('Invalid board position')
17        if self._board[i][j] != ' ':
18            raise ValueError('Board position occupied')
19        if self.winner() is not None:
20            raise ValueError('Game is already complete')
21        self._board[i][j] = self._player
22
23        if self._player == 'X':
24            self._player = 'O'
25        else:
26            self._player = 'X'
27
28    def _is_win(self, mark):
29        ''' Check whether the board configuration is a win for a given player'''
30        board = self._board
31
32        return (mark == board[0][0] == board[0][1] == board[0][2] or # row 0
33                mark == board[1][0] == board[1][1] == board[1][2] or # row 1
34                mark == board[2][0] == board[2][1] == board[2][2] or # row 2
35                mark == board[0][0] == board[1][0] == board[2][0] or # column 0
36                mark == board[0][1] == board[1][1] == board[2][1] or # column 1
37                mark == board[0][2] == board[1][2] == board[2][2] or # column 2
38                mark == board[0][0] == board[1][1] == board[2][2] or # diagonal
39                mark == board[0][2] == board[1][1] == board[2][0]) # rev diag
40
41    def winner(self):
42        ''' return mark of the winning player, or None to indicate a tie'''
43        for mark in 'XO':
44            if self._is_win(mark):
45                return mark
46        return None
47
48    def __str__(self):
49        ''' return th estring representation of the current game board'''
50        rows = ['|'.join(self._board[r]) for r in range(3)]
51        return '\n-----\n'.join(rows)
52

```

```

53
54 game = TicTacToe()
55
56 # X moves          # O moves
57 game.mark(1,1);    game.mark(0,2)
58 game.mark(2,2);    game.mark(0,0)
59 game.mark(0,1);    game.mark(2,1)
60 game.mark(1,2);    game.mark(1,0)
61 game.mark(2,0);
62
63 print(game)
64 winner = game.winner()
65 if winner is None:
66     print('Tie')
67 else:
68     print(winner, 'wins')
69

```

```

O|X|O
-----
O|X|X
-----
X|O|X
Tie

```

随堂练习：通用的获胜判定

- 改写 `_is_win(mark)`，不要写死下标，改为对二维数组循环。
- 解法应当：
 - 检查所有行
 - 检查所有列
 - 检查两条对角线
- 扩展（选做）
 - 改成对任意棋盘大小 N 都适用

小结

- 本讲我们学习了：
 - 底层数组的工作方式：初始化、访问元素等
 - 动态数组的创建与管理
 - 多维数组的创建